Supplementary Topics on Testing

Minsoo Ryu

Hanyang University

Topics Covered

1. Test-Driven Development
2. Test Coverage
3. Risk-based Testing
4. Mutation Analysis
5. Automated Testing

Mutation Analysis

- Mutation analysis
  - Modify programs' source code or byte code and then run a test suite
- Purpose of mutation analysis
  - Determine whether the tests are correct and sufficiently cover the requirements that have originated the implementation

Overview

- Mutation is done by selecting a set of mutation operators and then applying them to the source program one at a time for each applicable piece of the source code

- The result of applying one mutation operator to the program is called a mutant

- If the test suite is able to detect the change (i.e. one of the tests fails), then the mutant is said to be killed

Example

- Before mutation

```
if (a && b) {
    c = 1;
} else {
    c = 0;
}
```

- After mutation

```
if (a || b) {
    c = 1;
} else {
    c = 0;
}
```

→ mutant

“a=1 and b=0 will kill the mutant”

Weak and Strong Mutation Testing

- **Two conditions**
  - **Test input data should cause different program states for the mutant and the original program**
    - For example, a test with a = 1 and b = 0 would do this
  - **The value of 'c' should be propagated to the program's output and checked by the test**

- **Weak mutation testing (or weak mutation coverage) requires that only the first condition is satisfied**

- **Strong mutation testing requires that both conditions are satisfied**
  - **Strong mutation is more powerful, since it ensures that the test suite can really catch the problems**

Mutation Operators

- Traditional mutation operators for imperative languages:
  - Statement deletion
  - Replace each boolean subexpression with *true* and *false*
  - Replace each arithmetic operation with another one, e.g. + with *, - and /
  - Replace each boolean relation with another one, e.g. > with >=, == and <=
  - Replace each variable with another variable declared in the same scope (variable types should be the same)

- There are mutation operators for object-oriented languages, for concurrent constructions, complex objects like containers
  - They are called class-level mutation operators

Equivalent Mutants

- It may not possible to find a test case that could kill some mutants
  - Such mutants are called equivalent mutants

- Equivalent mutants detection is one of biggest obstacles for practical usage of mutation testing
  - The effort needed to check if mutants are equivalent or not, can be very high even for small programs

Equivalent Mutants

❒ Before mutations

```
int index = 0;

while (…)
{
    …;
    index++;

    if (index == 10) {
        break;
    }
}
```

❒ After mutation

```
int index = 0;

while (…)
{
    …;
    index++;

    if (index >= 10) {
        break;
    }
}
```

Topics Covered

1. Test-Driven Development
2. Test Coverage
3. Risk-based Testing
4. Mutation Analysis
5. Automated Testing

Manual Testing

- **Test cases are lists of instructions**
  - **"test scripts"**
- **Someone manually executes the script**
  - **Do each action, step-by-step**
    - Click on "login"
    - Enter username and password
    - Click "OK"
    - …
  - **And manually records results**
- **Low-tech, simple to implement**

Test Script Example

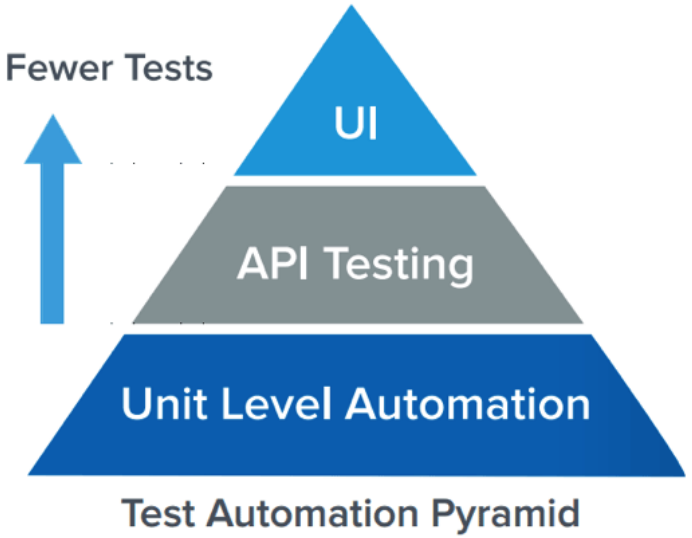
Test-case Identifier:	TC-1
Use Case Tested:	UC-1, main success scenario, and UC-7
Pass/fail Criteria:	The test passes if the user enters a key that is contained in the database, with less than a maximum allowed number of unsuccessful attempts
Input Data:	Numeric keycode, door identifier

Test Procedure:	**Expected Result:**
Step 1. Type in an incorrect keycode and a valid door identifier	System beeps to indicate failure; records unsuccessful attempt in the database; prompts the user to try again
Step 2. Type in the correct keycode and door identifier	System flashes a green light to indicate success; records successful access in the database; disarms the lock device

Automated Testing

- Automated testing makes use of specialized tools to control the execution of tests and compare the actual results against the desired results

- Test automation pyramid
  - UI testing
  - API testing
  - Unit testing



Test Automation Pyramid

What Can Be Automated?

- Test generation
  - Test case (steps with data & oracle)
  - Test data (input)
  - Test oracle (expected output)
  - Test verdict (PASS/FAIL decision)
- Test execution
  - E.g., regression testing

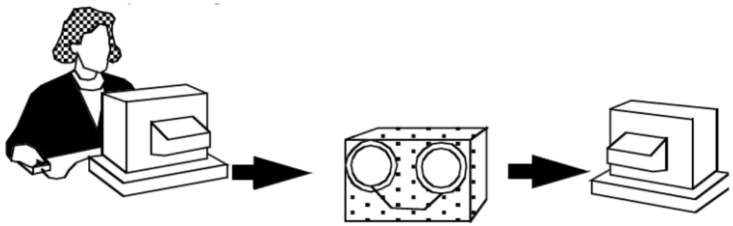


Often, people mean automated test execution when they talk about test automation

- Test reporting
- Debugging
  - Fault localization (using failure/error information)

Recorded Scripts

- Scripts are generated using capture and replay tools
  - Record user's actions to a script (keyboard, mouse)
  - Scripts access the user interface of the software (input fields, buttons and other widgets)



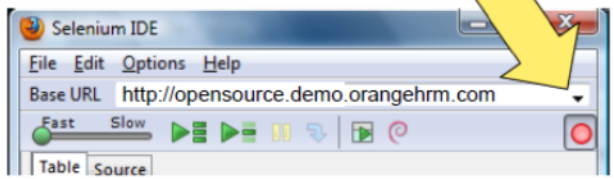
- Advantages
  - Relatively quick to set up
  - Mostly used for regression testing

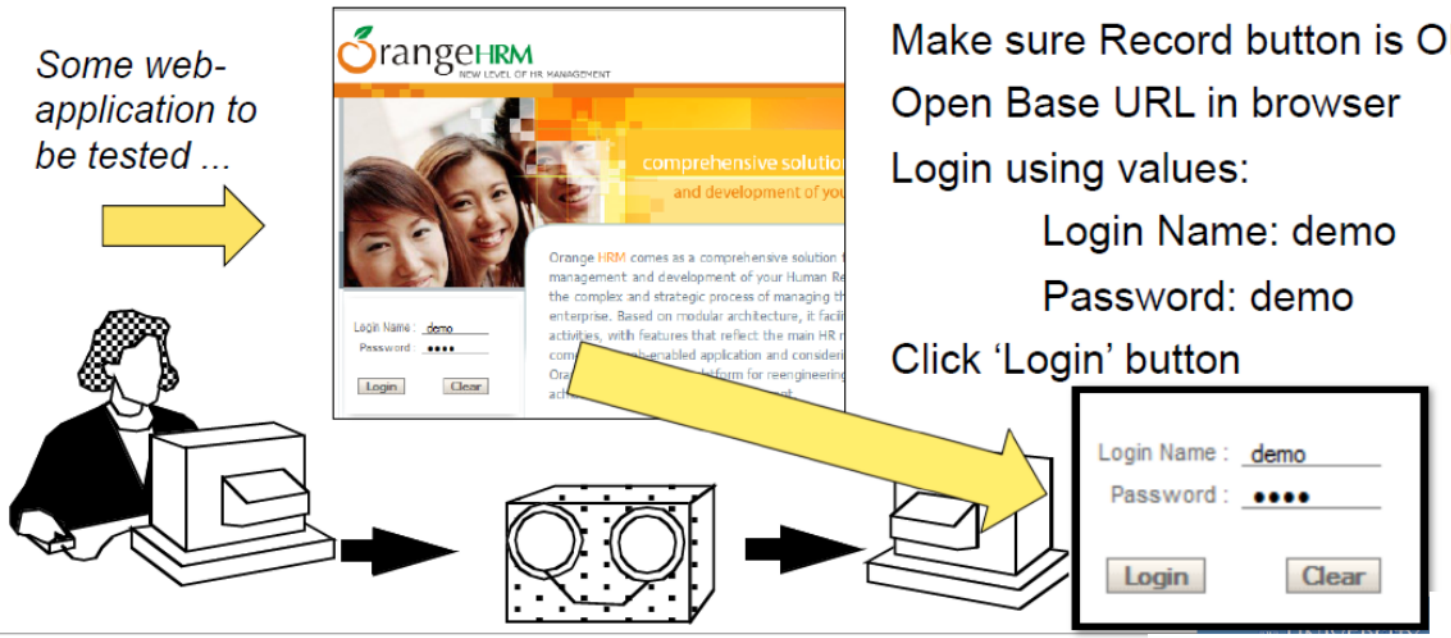
- Disadvantages
  - Scripts non-maintainable, in practice
  - Test recording is usually very fragile
    - If anything changes, the test breaks
    - Example) location, background color of textbox, the name of a dialog box, …

Example 1/2

Recorded Scripts

- Example with Selenium IDE





Example 2/2

Recorded Scripts

- Example with Selenium IDE

Test Case (=Test Scenario) consists of several Test Steps

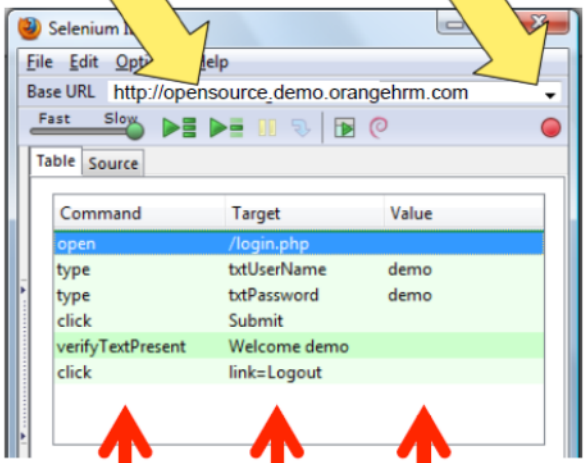
Test Suite
- TC1
- TC2
  - Step1
  - Step2 ...

Open Base URL in browser
Login using values:
Login Name: demo
Password: demo
Click Login button
Highlight 'Welcome demo' text
Verify that text is present
Click 'Logout'

TCs can be saved and exported into several programming languages (java, python, c#, etc.)

Tests can be run (replay) ...

Record Button switched off ...



Selenium commands

Data (values)

Location on Web-page (Target)
may use xpath, css, id, field, etc.

Engineered Scripts

- **Recorded tests are too low level**
  - **E.g., every test contains the name of the dialog box**
- **Need to abstract tests**
  - **Replace dialog box string by variable name X**
  - **Variable name X is maintained in one place**
    - So that when the dialog box name changes, only X needs to be updated and all the tests work again
- **Test scripts are mostly written manually**
  - **Scripts should be well-designed (following a systematic approach), modular, robust, documented, and maintainable**
    - Separation of common tasks: setup, cleanup/teardown, and defect detection
  - **Implementation and maintenance require programming skills which testers (test engineers) might not have**
    - This is just structured programming
  - **Test data is still embedded into the scripts**
    - One driver script per test case

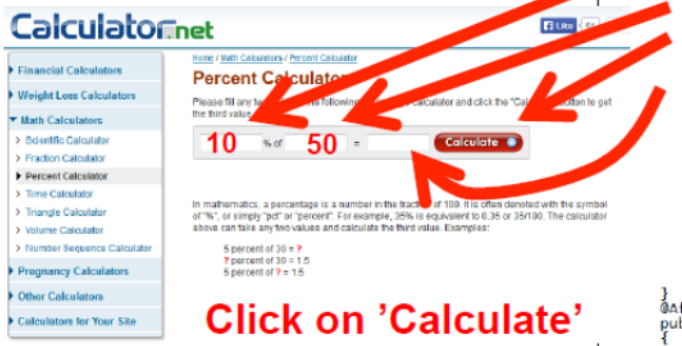
Example

Engineered Scripts

- Example with Selenium / TestNG

Enter ’10’ – first number

Enter ’50’ – second number



Click on ’Calculate’

Get result with getText() – ’5’

```
package TestNG;

import java.util.concurrent.TimeUnit;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;

import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;

public class TestNGClass
{
   WebDriver driver = new FirefoxDriver();

   @BeforeTest
   public void launchapp()
   {
      // Puts an Implicit wait, Will wait for 10 seconds before throwing exception
      driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
      // Launch website
      driver.navigate().to("http://www.calculator.net");
      driver.manage().window().maximize();
   }
   @Test
   public void calculatepercent()
   {
      // Click on Math Calculators
      driver.findElement(By.xpath(".//*[@id='menu']/div[3]/a")).click();

      // Click on Percent Calculators
      driver.findElement(By.xpath(".//*[@id='menu']/div[4]/div[3]/a")).click();

      // Enter value 10 in the first number of the percent Calculator
      driver.findElement(By.id("cpar1")).sendKeys("10");

      // Enter value 50 in the second number of the percent Calculator
      driver.findElement(By.id("cpar2")).sendKeys("50");

      // Click Calculate Button
      driver.findElement(By.xpath(".//*[@id='content']/table/tbody/tr/td[2]/input")).click();

      // Get the Result Text based on its xpath
      String result = driver.findElement(By.xpath(".//*[@id='content']/p[2]/span/font/b")).getText();

      // Print a Log In message to the screen
      System.out.println(" The Result is " + result);

      if(result.equals("5"))
      {
         System.out.println(" The Result is Pass");
      }
      else
      {
         System.out.println(" The Result is Fail");
      }
   }
   @AfterTest
   public void terminatetest()
   {
      driver.close();
   }
}
```

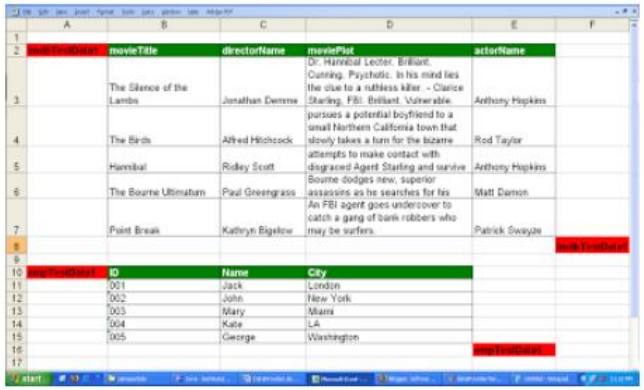
Data-Driven Testing

❒ Data-Driven Testing

- **Executing the same set of test steps with multiple (different) data**
- **One driver script can execute all of the designed test cases**

❒ Test inputs and expected outcomes stored as data

- **Typically in tabular format**
- **Test data are read from an external data source**



	A	B	C	D	E	F
1						
2		movieTitle	directorName	moviePlot	actorName	
3		The Silence of the Lambs	Jonathan Demme	Dr. Hannibal Lecter. Brilliant. Cunning. Psychotic. In his mind lies the clue to a ruthless killer. - Clarice Starling, FBI. Brilliant. Vulnerable.	Anthony Hopkins	
4		The Birds	Alfred Hitchcock	pursues a potential boyfriend to a small Northern California town that slowly takes a turn for the bizarre	Rod Taylor	
5		Hannibal	Ridley Scott	attempts to make contact with disgraced Agent Starling and survive	Anthony Hopkins	
6		The Bourne Ultimatum	Paul Greengrass	Bourne dodges new, superior assassins as he searches for his	Matt Damon	
7		Point Break	Kathryn Bigelow	An FBI agent goes undercover to catch a gang of bank robbers who may be surfers.	Patrick Swayze	
8						
9						
10		ID	Name	City		
11		001	Jack	London		
12		002	John	New York		
13		003	Mary	Miami		
14		004	Kate	LA		
15		005	George	Washington		
16						
17						

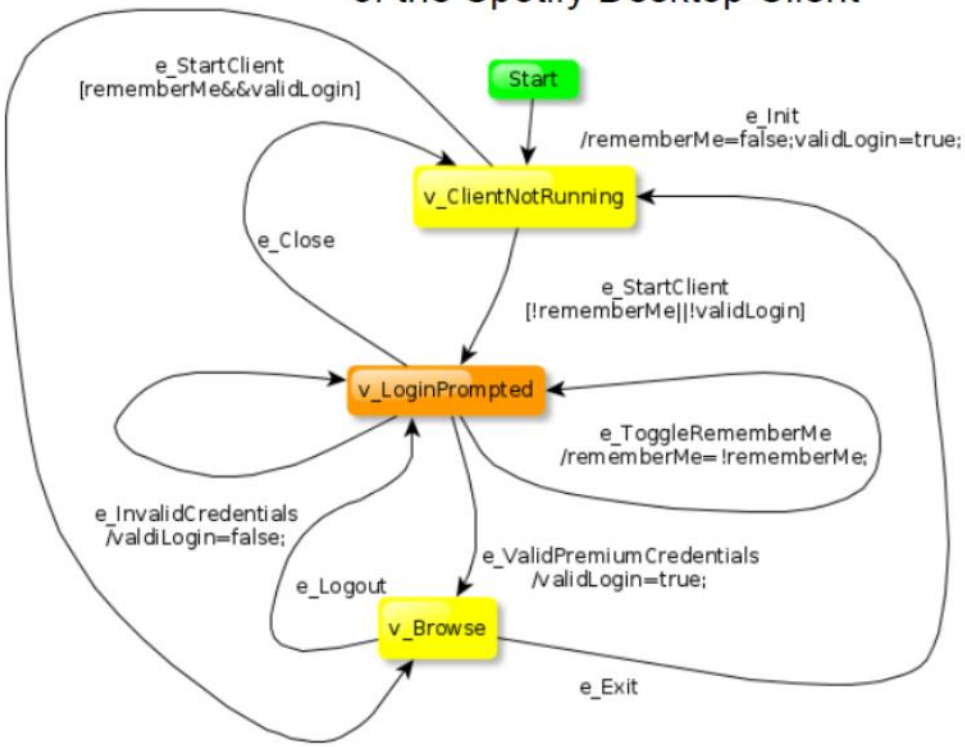
Model-based Testing

- System under test is modelled
  - UML state machines, domain specific languages (DSL)
- Test cases are automatically generated from the model
  - The model can provide also the expected results for the generated test cases
  - More accurate model → better test cases
- Generate a large amount of tests that cover the model
  - Many different criteria for covering the model
  - Execution time of test cases might be a factor
- Challenges
  - Personnel competencies
  - Data-intensive systems (cannot be modelled as a state-machine)

Example

Example:

A regression test for the login function of the Spotify Desktop Client




```
%> java -jar
graphwalker.jar offline -m
Login.graphml
"random(edge_coverage(100))
"
e_Init v_ClientNotRunning
e_StartClient
v_LoginPrompted
e_InvalidCredentials
v_LoginPrompted
e_ValidPremiumCredentials
v_Browse e_Logout
v_LoginPrompted e_Close
v_ClientNotRunning
...
```

Manual Testing vs. Automated Testing

- **Manual testing is widespread**
  - **Maybe not dominant, but very common**

- **Why? Because**
  - **Some tests can't be automated**
    - Usability testing
  - **Some tests shouldn't be automated**
    - Not worth the cost

Manual Testing vs. Automated Testing

- **Automated testing does not replace manual testing**
  - **Manual testing has hidden strength**
    - For example, because people are doing the tests, it is easy to adapt tests to slightly modified situations
  - **Manual tests find more defects than automated tests**
    - Automation may improve efficiency, but, not effectiveness

- **Maintaining tests is a lot of work**
  - **Broken tests must be fixed; this is expensive**
  - **Cost is proportional to the number of tests**
  - **Implies that more tests is not necessarily better**


Q & A

Misconceptions About Automated Testing

- "Automation will provide you with more free time"
  - Both true and false
  - In manual testing, most of the time is devoted to exploratory and functional testing where you would manually search for errors
  - With automated testing, the work for automated testers is instead spent coding the tests and making improvements to these tests repeatedly as adjustments are needed
  - Once the test is complete, however, automated testing allows for the recycled use of tests so that they do not have to go through this whole process again

Misconceptions About Automated Testing

- "The Cost of Automated Testing is Too High"
  - At first, the investment in automated testing might seem cost prohibitive, especially if you're a smaller company
- But analysis has shown that, over time, automated testing pays for itself
  - Automated testing frees you up to focus on larger issues such as customer needs, functionality and improvements
  - Automated testing also reduces the cost and need for multiple code revisions, so over the course of time, the investment pays out
  - In addition, each time the source code is modified, the software tests can be repeated
  - Manually repeating these tests is costly and time-consuming, but automated tests can be run over and over again at no additional cost

Misconceptions About Automated Testing

- "Automated Testing is Better Than Manual Testing"
  - There is no "better" or "worse" in the automated vs. manual debate, there's just "different"
  - Each approach has its own advantages and disadvantages
  - Both manual and automated testing have their roles, especially if the software you're developing is too large and too complex to rely just on the manual approach

- "Automated Testing Inhibits Human Interaction"
  - It doesn't replace the face-to-face communication that's a necessary part of software development
  - Instead, it enhances that aspect by providing another channel through which to communicate
    - Email didn't replace the telephone